

1 Introduction

2 Typed Assembly Languages

Typed assembly languages (TAL) define some manner of types to an assembly program. Assembly languages are low-level, so the degree of typing, or where these types even occur may not be obvious. Instructions in assembly work on words and immediate values, and these instructions are strictly distinct with no overloading—¹ so the notion of types here seems unneeded. Indeed instructions are not typed in TALs, but labels are. One simple implementation of this—which types registers—is Morrisett's TAL-0. Morrisett extends this language to work on memory (TAL-0), and ??? extends types with dependently typed assembly (DTAL).

The motivation for using TALs is commonly attributed to safety and correctness.

2.1 TAL-0

Label types contain information on what types are contained within relevant registers and memory addresses.

A well-formed typed assembly program contains no references to values that are not within one such type(wrong!)—and does not violate them if one such exists.

Given some types t of data

$$t ::= \{r_1 : t_1, \dots, r_n : t_n\} \quad (1)$$

allows us to assign types to registers following a label, so

```
; push (r : 'T) (sp : 'T::[...]) : 'T::'T::[...]  
; pop (sp : 'T::[...]) : 'T
```

```
fib: {a1 : int, sp : [sp :: int], ra : {a0 : int}}  
    addi    a1, a1, -1  
    bltz    ...
```

```
base: {...}  
    addi    a1, zero, 1
```

end:

is a function that takes on ints, and jumps to a place where a0 is expected to be an int. We also type the stack to contain this, as well (since we will be pushing and popping for the recursive calls). This makes sure that fib pushes and pops the right type.

```
ouch: {a1 : int}  
    jmp     a1
```

$$\forall \alpha, \beta. \{r_1 : \alpha, r_2 : \beta, r_3 : \{r_1 : \beta, r_2 : \alpha\}\} \quad (2)$$

might describe a label that swaps the values of two registers.

Most commonly used in correctness analysis and ..

¹not AI

2.2 abstract Machine

2.3 Judgements / rules

use lit/tal.pdf slide 22 show well typed

3 Implementation

Using dependently typed assembly language based on [OGI], ...

3.1 Theory (tentative title)

3.1.1 Abstract machine

This project focuses on `ssd_os` and the machine that it runs on (or perhaps will be run on) is a realtime system, and has no cache, but two states of memory; scratch and shared. Scratch is the fastest one, and acts like manual cache.

3.1.2 Extended DTAL (and possible exclusions)

The only extension to DTAL that is needed is to type which kind of memory is used; scratch or shared. To this end, we need the constructors

$$\epsilon :: = \text{scratch } \epsilon \quad (3)$$

$$| \text{shared } \epsilon \quad (4)$$

this makes it possible to create, for instance, `scratch  $\alpha$  array`.

3.2 Code (tentative title)